

# Programmieren 1

---



## 02: Operatoren & Ausdrücke

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

[simon.dewert@haw-hamburg.de](mailto:simon.dewert@haw-hamburg.de)

# Beispiele der Abgaben

---

```
System.out.println("Das Ergebnis lautet: \n" +  
    ersteZahl + " + " + zweiteZahl + " = " + String.format("%.1f ", (ersteZahl + zweiteZahl)) + "\n" +  
    ersteZahl + " - " + zweiteZahl + " = " + String.format("%.1f ", (ersteZahl - zweiteZahl)) + "\n" +  
    ersteZahl + " * " + zweiteZahl + " = " + String.format("%.2f ", (ersteZahl * zweiteZahl)) + "\n" +  
    ersteZahl + " / " + zweiteZahl + " = " + String.format("%.2f ", (ersteZahl / zweiteZahl)));
```

```
wert3 = wert1 / wert2;  
System.out.printf("\n%.1f + %.1f = %.2f", wert1, wert2, wert3);
```

# Übersicht dieser Lehreinheit

---

1. Operatoren-Topologie
2. Mathematische Operatoren
3. Relationale Operatoren
4. Logische Operatoren
5. Bitweise Operatoren

# Operatoren-Topologie

| Operatoren    |            |               |          |                                       |
|---------------|------------|---------------|----------|---------------------------------------|
| Mathematisch  | Relational | Logisch       | Bitweise | Sonstige                              |
| Arithmetik    |            | Full Circuit  | Logisch  | Concat<br><code>Str1 + Str2</code>    |
| Zuweisung     |            | Short Circuit | Shift    | Cast<br><code>(int)</code>            |
| In-/Dekrement |            |               |          | Ternär<br><code>? :</code>            |
| Postfix       |            |               |          | Instanceof<br><code>instanceof</code> |
| Präfix        |            |               |          |                                       |

# Arithmetische Operatoren

Mit arithmetischen Operatoren werden Grundrechenarten ausgeführt.

| Operator | Bedeutung                  | Beispiel                                                                                                                                               |
|----------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| +        | Addition                   | <code>int x = 3 + 4;           // x = 7</code>                                                                                                         |
| -        | Subtraktion                | <code>int x = 3 - 4;           // x = -1</code>                                                                                                        |
| *        | Multiplikation             | <code>int x = 3 * 4;           // x = 12</code>                                                                                                        |
| /        | Division                   | <code>int x = 7 / 4;           // x = 1</code><br><code>double y = 7.0 / 4.0;   // y = 1.75</code><br><code>double z = 7.0 / 4;     // z = 1.75</code> |
| %        | Modulo<br>(Divisions-Rest) | <code>int x = 11 % 4;          // x = 3</code><br><code>double y = 11.0 % 4.0;   // y = 3.0</code><br><code>double z = 11.0 % 4;     // z = 3.0</code> |

es gilt:  
Punkt vor  
Strich

$11 / 4 = 2,75$   
 $4 * 2 = 8$   
 $11 - 8 = 3$

Darüber hinaus bietet die Klasse **Math** viele zusätzliche Rechenoperationen an.  
Siehe: <https://docs.oracle.com/javase/10/docs/api/java/lang/Math.html>

# Aufgabe 2.1



Legen Sie zunächst ein neues Java-Projekt namens **vorlesung\_02** an!  
Schreiben Sie dann ein Programm **MatheFunktion**. In diesem Programm sollen unterschiedliche mathematische Funktionen mit Hilfe des Matheobjekts **Math** durchgeführt werden.

*Hinweis: Tippen Sie **Math**. ein, um die verfügbaren Funktionen zu sehen.*

## Beispiel:

Geben Sie eine beliebige Ganzzahl ein:  
Geben Sie eine weitere Ganzzahl ein:

2  
-5

Eingabe

Die Absolutwerte sind: 2, 5  
Die kleinere Zahl ist: -5  
Beide Zahlen multipliziert ergeben: -10

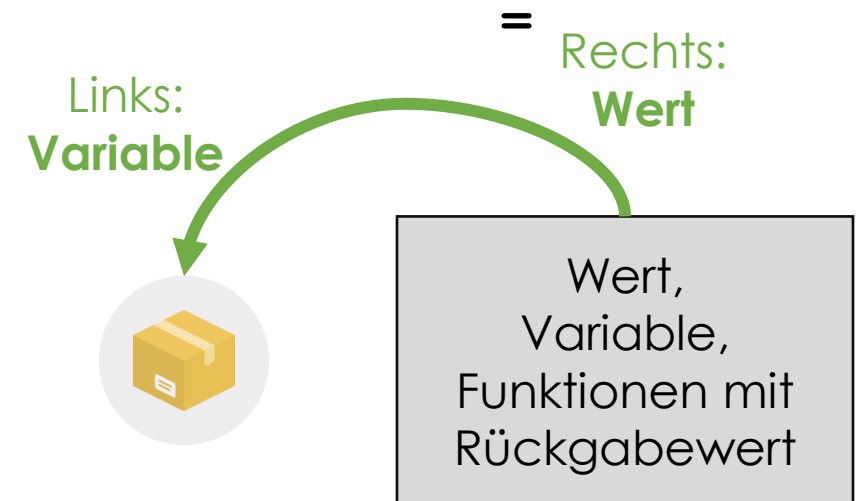
Ausgabe

# Zuweisungs-Operator

Der Zuweisungsoperator **=** weist dem linken Operanden den Wert des rechten Operanden zu und ist für alle elementaren Datentypen definiert.

Dabei kann der rechte Operand ein Literal, eine Variable oder ein Ausdruck (Bsp.: eine arithmetische Operation oder der Aufruf einer Methode mit Rückgabewert) sein.

```
int x = 3;  
int y = x;           // y = 3  
int z = x + y;       // z = 6  
int f = someFunction();
```



# Verbundzuweisungs-Operatoren

Verbundzuweisungs-Operatoren stellen eine Kombination eines arithmetischen Operators mit einem Zuweisungs-Operator dar:

```
int x = 3;  
x += 4;      // gleichbedeutend mit x = x + 4;  
x -= 5;      // gleichbedeutend mit x = x - 5;  
x *= 6;      // gleichbedeutend mit x = x * 6;  
x /= 7;      // gleichbedeutend mit x = x / 7;  
x %= 8;      // gleichbedeutend mit x = x % 8;
```

Frage: Wie groß ist x am Ende dieser Folge von Operationen?



# In-/Dekrement-Operatoren

Der **Inkrement**-Operator **++** **erhöht** den Wert der Variable um 1.

Der **Dekrement**-Operator **--** **verringert** den Wert einer Variable um 1.

Beide Operatoren gibt es in einer Präfix- und Postfix-Notation:

```
int x = 2, y = 3;  
x = ++y;      // Präfix (nach dieser Zeile ist x = 4 und y = 4)  
y = x++;      // Postfix (nach dieser Zeile ist x = 5 und y = 4)
```

→ Präfix: Erhöhung des Operanden und Rückgabe des **neuen** Werts

→ Postfix: Erhöhung des Operanden und Rückgabe des **alten** Werts

# Aufgabe 2.2



Erstellen Sie im Projekt **vorlesung\_02** ein Programm **TemperaturUmrechnung**, das einen eingegebenen Wert auf zwei Nachkommastellen genau in Grad Fahrenheit (°F) und Kelvin (K) umrechnet!

*Hinweis: Erkundigen Sie sich ggf. im Internet, wie die Umrechnung erfolgt!*

Beispiel:

Geben Sie eine Temperatur in Grad Celsius ein:

22

Eingabe

22°C entsprechen 71.6°F und 295.15K.

Ausgabe

# Operatoren-Topologie

|               | Operatoren |               |          |                                       |
|---------------|------------|---------------|----------|---------------------------------------|
| Mathematisch  | Relational | Logisch       | Bitweise | Sonstige                              |
| Arithmetik    |            | Full Circuit  | Logisch  | Concat<br><code>Str1 + Str2</code>    |
| Zuweisung     |            | Short Circuit | Shift    | Cast<br><code>(int)</code>            |
| In-/Dekrement |            |               |          | Ternär<br><code>? :</code>            |
| Postfix       |            |               |          | Instanceof<br><code>instanceof</code> |
| Präfix        |            |               |          |                                       |

# Relationale Operatoren

Relationale Operatoren dienen dem Vergleich **zweier Zahlen**. Das Ergebnis einer relationalen Operation ist vom Datentyp **boolean** und somit entweder wahr (**true**) oder falsch (**false**).

```
x < y;           // ist x kleiner y?  
x <= y;          // ist x kleiner oder gleich y?  
x == y;          // ist x gleich y?  
x > y;           // ist x größer y?  
x >= y;          // ist x größer oder gleich y?  
x != y;          // ist x ungleich y?
```

**Achtung:** Beim Vergleichen von **Strings** ist die Methode **.equals()** zu verwenden!

*Hinweis: Relationale Operatoren werden wichtig im Kapitel Verzweigungen!*

# Unterschied: = und ==

---

Wie auf Folie 7 beschrieben, handelt es sich beim einfachen = um den Zuweisungsoperator.

Auf Folie 12 haben wir die doppelten == als Vergleichsoperator kennengelernt.

=      **Zuweisungsoperator** (Befüllt Variablen)

==      **Vergleichsoperator** (Vergleicht Werte, Variablen uws.)

# Aufgabe 2.3



Erstellen Sie im Projekt **vorlesung\_02** ein Java-Programm namens **GeradeZahlenChecker**, das Sie dazu auffordert, eine beliebige ganze, gerade Zahl einzugeben!

Lassen Sie das Programm (ohne Kontrollstruktur!) ein **einfaches Feedback** (**true** oder **false**) geben, ob die Zahl tatsächlich gerade ist!

Beispiel:

Geben Sie eine beliebige, gerade Ganzzahl ein:

7

Eingabe

false

Ausgabe

# Operatoren-Topologie

| Operatoren    |            |               |          |                                       |
|---------------|------------|---------------|----------|---------------------------------------|
| Mathematisch  | Relational | Logisch       | Bitweise | Sonstige                              |
| Arithmetik    |            | Full Circuit  | Logisch  | Concat<br><code>Str1 + Str2</code>    |
| Zuweisung     |            | Short Circuit | Shift    | Cast<br><code>(int)</code>            |
| In-/Dekrement |            |               |          | Ternär<br><code>? :</code>            |
| Postfix       |            |               |          | Instanceof<br><code>instanceof</code> |
| Präfix        |            |               |          |                                       |

# Logische Operatoren

Logische Operatoren dienen der **Auswertung zweier boolescher** Ausdrücke im Sinne der UND-, ODER- und NICHT-Logik.

x und y werden hier als  
Variablen vom Typ **boolean**  
angenommen

|                                         |                                                                                                                       |                              |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------|
| <code>x &amp; y</code>                  | <code>// true, wenn x UND y true sind</code>                                                                          | <b>UND (Full Circuit)</b>    |
| <code>x &amp;&amp; y</code>             | <code>// true, wenn x UND y true sind</code><br><code>// Abbruch mit false, wenn bereits x false ist</code>           | <b>UND (Short Circuit)</b>   |
| <code>x   y</code>                      | <code>// true, wenn x ODER y ODER beide true sind</code>                                                              | <b>ODER (Full Circuit)</b>   |
| <code>x    y</code>                     | <code>// true, wenn x ODER y ODER beide true sind</code><br><code>// Abbruch mit true, wenn bereits x true ist</code> | <b>ODER (Short Circuit)</b>  |
| <code>x ^ y</code>                      | <code>// true, wenn x und y UNGLEICH sind</code>                                                                      | <b>Exklusives ODER (XOR)</b> |
| <code>!x</code>                         | <code>// true, wenn x false und umgekehrt</code>                                                                      | <b>NICHT (Negation)</b>      |
| <code>x == 1 &amp;&amp; y &gt; 2</code> | <code>// true, wenn x = 1 und y größer 2</code>                                                                       | <b>Beispiel</b>              |



# Aufgabe 2.4



Erstellen Sie im Projekt **vorlesung\_02** ein Java-Programm namens **WerteBereichsChecker**, das Sie dazu auffordert, eine beliebige ganze Zahl einzugeben, die **zwischen zwei Zufallszahlen (0,100)** liegt.

Lassen Sie das Programm (ohne Kontrollstruktur!) ein **einfaches Feedback** (**true** oder **false**) geben, ob eine geeignete Zahl eingegeben wurde

Beispiel:

Geben Sie eine beliebige Ganzzahl zwischen 15 und 27 ein: **7** **Eingabe**

**false** **Ausgabe**

*Alle die vorzeitig fertig werden, können das Programm so anpassen, dass die Grenzwerte auch verwendet werden dürfen.*

# Zufallszahl in Java

---

Hier zwei Möglichkeiten eine Zufallszahl in Java zu generieren.

**Math.random();**

Liefert eine zufällige Gleitkommazahl zwischen 0 und 1.0

**Random – Klasse**

Eine Klasse extra für Zufallswerte. Hier gibt es mehr Möglichkeiten den Wertebereich festzulegen, aber das Handling ist für Anfänger schwerer zu verstehen.

```
Random r = new Random();
```

```
int zahl = r.nextInt(10);
```

# Operatoren-Topologie

| Operatoren    |            |               |          |                                       |
|---------------|------------|---------------|----------|---------------------------------------|
| Mathematisch  | Relational | Logisch       | Bitweise | Sonstige                              |
| Arithmetik    |            | Full Circuit  | Logisch  | Concat<br><code>Str1 + Str2</code>    |
| Zuweisung     |            | Short Circuit | Shift    | Cast<br><code>(int)</code>            |
| In-/Dekrement |            |               |          | Ternär<br><code>? :</code>            |
| Postfix       |            |               |          | Instanceof<br><code>instanceof</code> |
| Präfix        |            |               |          |                                       |

# Logische Bitweise Operatoren 1/2

Logische Bitweise Operatoren erlauben logische Operationen **auf Bitebene** der Datentypen **char**, **byte**, **short**, **int** und **long**.

## Die Operatoren UND und ODER auf Bitebene:

sog. „Bitmuster“ bzw. hier „8-Bit-Muster“

```
byte x = 0, y = 1, z = 2;
```

```
x = y & z;          // Bitweises UND:  0000 0001
                                     & 0000 0010  (x = 0)
                                     -----
                                     0000 0000
```

```
x = y | z;          // Bitweises ODER:  0000 0001
                                     | 0000 0010  (x = 3)
                                     -----
                                     0000 0011
```

# Logische Bitweise Operatoren 2/2

Logische Bitweise Operatoren erlauben logische Operationen **auf Bitebene** der Datentypen `char`, `byte`, `short`, `int` und `long`.

## Die Operatoren XOR und Negation (Komplement) auf Bitebene:

sog. „Bitmuster“ bzw. hier „8-Bit-Muster“

```
byte x = 0, y = 2, z = 3;
```

```
x = y ^ z;           // Bitweises XOR:  0000 0010  
                                ^ 0000 0011  (x = 1)  
                                0000 0001
```

```
x = ~y;             // Bitweises Komplement:  ~ 0000 0010  (x = -3)  
                                1111 1101
```

Vorzeichen-Bit

Bitweises Komplement → „Negation +(-1)“

# Exkurs: 2er-Komplement

Integer-Datentypen werden im sogenannten 2er-Komplement codiert, um unter **Wahrung der Rechenregeln** sowohl **negativ** als auch **positive** Zahlen darstellen zu können.

Bsp.: Datentype **byte** (8 Bit):

|                 |      | Zahlwert-Bits |    |    |    |   |   |   |   |
|-----------------|------|---------------|----|----|----|---|---|---|---|
| 2er-Komplement: |      | -128          | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| Beispiel:       | +3 → | 0             | 0  | 0  | 0  | 0 | 0 | 1 | 1 |
| Beispiel:       | -3 → | 1             | 1  | 1  | 1  | 1 | 1 | 0 | 1 |

} Summe ergibt 0 ✓

0000 0000 dez 0  
1111 1111 dez -1

# Exkurs: 2er-Komplement

---

## Eigenschaften

- Das höchstwertige Bit (Most significant Bit, MSB) zeigt das Vorzeichen an (0 für positiv, 1 für negativ).
- Es gibt nur **eine** Darstellung für die Null (0000 0000).
- Addition und Subtraktion von Zahlen funktionieren einheitlich, ohne spezielle Regeln für Vorzeichen.

Das Zweierkomplement ermöglicht eine einfache Handhabung von arithmetischen Operationen und ist daher in Computer weit verbreitet.

# Exkurs: 2er-Komplement

1. Positive Zahlen: werden wie üblich im Binärsystem dargestellt.
2. Negative Zahlen:
  - Nimm die binäre Darstellung der entsprechenden positiven Zahl
  - Invertiere alle Bits (0 wird zu 1, 1 wird zu 0).
  - Addiere 1 zum Ergebnis.

Beispiel:  $+3 =$

| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |   |   |   |   |   |   |   |
|------|----|----|----|---|---|---|---|---|---|---|---|---|---|---|
| 0    | 0  | 0  | 0  | 0 | 0 | 1 | 1 |   |   |   |   |   |   |   |
| 0    | +  | 0  | +  | 0 | + | 0 | + | 0 | + | 0 | + | 2 | + | 1 |

Beispiel:  $-3 =$

| -128 | 64 | 32 | 16 | 8  | 4 | 2  | 1 |   |   |   |   |   |   |   |
|------|----|----|----|----|---|----|---|---|---|---|---|---|---|---|
| 1    | 1  | 1  | 1  | 1  | 1 | 0  | 1 |   |   |   |   |   |   |   |
| -128 | +  | 64 | +  | 32 | + | 16 | + | 8 | + | 4 | + | 0 | + | 1 |



# Exkurs: 2er-Komplement

## Beispiel Subtraktion

Die Subtraktion im Zweierkomplement wird durch Addition des Negativen einer Zahl durchgeführt.

Um  $A - B$  zu berechnen, finde das Negative von  $B$  (siehe Folie 22) und addiere es zu  $A$ :  $A - B = A + (-B)$

Beispiel **7-5** im 8-Bit-System

**7**  $\leftrightarrow$  00000111;    **5**  $\leftrightarrow$  00000101;    finden von **-5**  $\leftrightarrow$  11111011

Addieren: 0000 0111

1 1111 1011

1 0000 0010

**Wichtig:** Bei Überlauf (Ergebnis hier 9 Bit statt 8) wird das überflüssige Bit ignoriert.

Ergebnis lautet also: 0000 0010  $\leftrightarrow$  **2**    (7-5 = 2)

# Bitweise Shift-Operatoren 1/3

**Shift-Operatoren „verschieben“ das Bitmuster einer Variablen** um eine anzugebende Anzahl an Stellen nach links oder rechts.

**Verschieben um n Stellen nach links: Multiplikation mit  $2^n$**

```
int x = 0, y = 3; // Verschieben um 2 Stellen nach LINKS
```

```
x = y << 2;
```

```
00000000 00000000 00000000 00000011
-> 00000000 00000000 00000000 00001100
(x = 12)
```

```
y = -3;
```

```
x = y << 2; // Verschieben um 2 Stellen nach LINKS
```

```
11111111 11111111 11111111 11111101
-> 11111111 11111111 11111111 11110100
(x = -12)
```

„Auffüllen“  
mit Nullen

# Bitweise Shift-Operatoren 2/3

**Shift-Operatoren „verschieben“ das Bitmuster einer Variablen** um eine anzugebende Anzahl an Stellen nach links oder rechts.

**Verschieben um n Stellen nach rechts: Division durch  $2^n$**

```
int x = 0, y = 12; // Verschieben um 2 Stellen nach RECHTS
```

```
x = y >> 2;
```

00000000 00000000 00000000 00001100  
-> 00000000 00000000 00000000 00000011  
(x = 3)

„Auffüllen“  
mit Nullen

```
y = -12;
```

```
x = y >> 2; // Verschieben um 2 Stellen nach RECHTS
```

11111111 11111111 11111111 11110100  
-> 11111111 11111111 11111111 11111101  
(x = -3)

„Auffüllen“ mit Einsen  
(Erhalt des Vorzeichen-Bits)

# Bitweise Shift-Operatoren 3/3

**Shift-Operatoren „verschieben“ das Bitmuster einer Variablen** um eine anzugebende Anzahl an Stellen nach links oder rechts.

**Verschieben um n Stellen nach rechts ohne Erhalt des Vorzeichen-Bits:**

```
int x = 0, y = 12; // Verschieben um 2 Stellen nach RECHTS
```

```
x = y >>> 2;
```

00000000 00000000 00000000 00001100  
-> 00000000 00000000 00000000 00000011  
(x = 3)

„Auffüllen“  
mit Nullen

```
y = -12;
```

```
x = y >>> 2; // Verschieben um 2 Stellen nach RECHTS
```

11111111 11111111 11111111 11110100  
-> 00111111 11111111 11111111 11111101  
(x = 1073741821)

„Auffüllen“ mit  
Nullen!

# Aufgabe 2.5



Erstellen Sie im Projekt **vorlesung\_02** ein Java-Programm namens **SchalterWahl**, in dem ein vorgegebenes **8-Bit-Muster** gezielt verändert werden kann.

Beispiel:

|                        |                      |         |
|------------------------|----------------------|---------|
| Anfangszustand:        | 10011010             | Ausgabe |
| Wähle Schalter (1-8) : | 5                    | Eingabe |
|                        | Änderung des 5. Bits |         |
| Endzustand:            | 10001010             | Ausgabe |

## Hinweis:

Erzeugung des Strings zur Ausgabe eines Integers als 8-Bit-Muster:

```
String.format("%8s", Integer.toBinaryString(4)).replace(' ', '0');
```

Zahl die als 8-Bit Muster angezeigt werden soll

# Operatoren-Topologie

| Operatoren    |            |               |          |                                       |
|---------------|------------|---------------|----------|---------------------------------------|
| Mathematisch  | Relational | Logisch       | Bitweise | Sonstige                              |
| Arithmetik    |            | Full Circuit  | Logisch  | Concat<br><code>Str1 + Str2</code>    |
| Zuweisung     |            | Short Circuit | Shift    | Cast<br><code>(int)</code>            |
| In-/Dekrement |            |               |          | Ternär<br><code>? :</code>            |
| Postfix       |            |               |          | Instanceof<br><code>instanceof</code> |
| Präfix        |            |               |          |                                       |

# Sonstige Operatoren

## Concat

Mit dem Concat-Operator lassen sich zwei Strings zusammenfügen.

```
String s = "Strings are immutable"  
s = s.concat(" all the time");
```

Kurzschreibweise: `"Strings are immutable" + " all the time"`

## Cast

Siehe Folien „Programmieren1\_02  
Seite 19

### Explizites Casting (Narrowing)

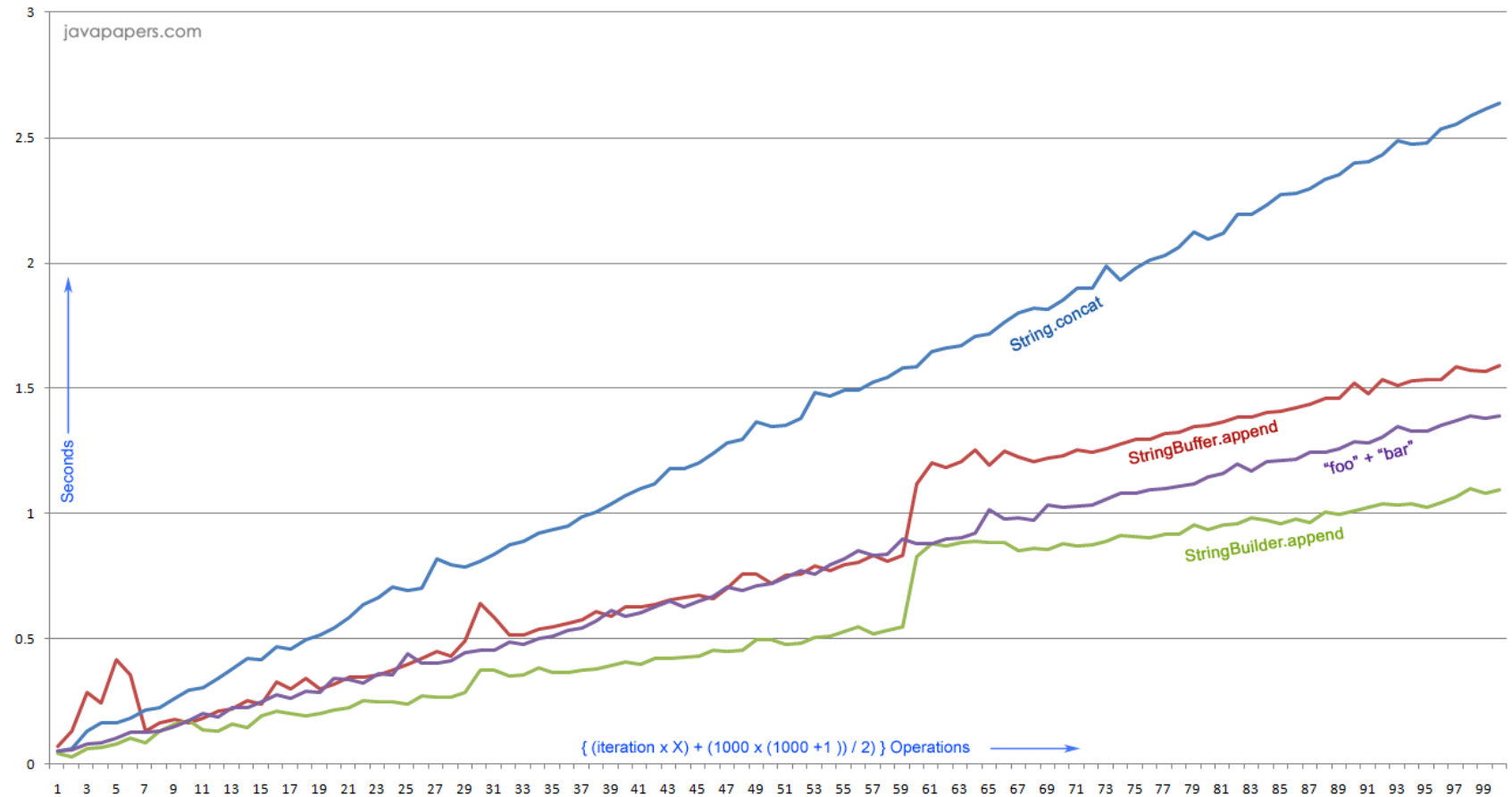
Wenn entgegen der Reihenfolge konvertiert werden soll, so muss der **Datentyp der Zielvariable** explizit in Klammern **vor der Quellvariable** angegeben werden:

```
meinByte = (byte) meinDouble;
```

Beachte:  
Der Wert der Quellvariable muss im zulässigen Wertebereich des Datentyps der Zielvariablen liegen!

# Concat - Performance

Vorsicht bei  
Verwendung  
von Concat in  
Schleifen  
(Einheit 5)





# Viele Dank für Ihre Aufmerksamkeit



# Hausaufgabe Folie 02

Erstellen Sie im Projekt **vorlesung\_02** ein Java-Programm namens **Personalien**, in welchem Personalien entsprechend des Beispiels eingeben werden können.

## Beispiel:

|               |            |
|---------------|------------|
| Name:         | John       |
| Nachname:     | Doe        |
| Geschlecht:   | männlich   |
| Geburtsdatum: | 01.01.1980 |
| Beruf:        | Müllmann   |

Eingabe

John Doe; männlich; geb. 01.01.1980; Müllmann

Ausgabe

Hinweis: Verwenden Sie wenn möglich **StringBuilder**  
(*google is the programmers friend*)